

using direct accesses to instance variables or using properties that Python translates into calls to decorated methods. I prefer to use standard getter and setter methods and will use them in the rest of this book.

Encapsulation in pygwidgets Classes

The definition of encapsulation at the start of this chapter focused on two areas: hiding internal details and having all related code in one place. All the classes in `pygwidgets` were designed with these considerations in mind. As examples, consider the `TextButton` and `CustomButton` classes.

The methods of these two classes encapsulate all the functionality of GUI buttons. While the source code of these classes is available, there is no need for a client programmer to look at it to use them effectively. There's also no need for client code to attempt to access any of their instance variables: all button functionality is available through calling the methods of these classes. This adheres to the strict interpretation of encapsulation, meaning that the *only* way that client software should access an object's data is by calling that object's methods. A client programmer can think of these classes as black boxes, since there is no reason to look at how they accomplish their tasks.

NOTE

A whole black box testing industry has developed around the idea of a test programmer being given a class to test without being allowed to see the code of the class. The tester is only supplied with documentation of the interfaces, and with that writes code that tests all the interfaces under many different cases to ensure that all methods work as described. The set of tests not only ensures that the code and documentation match, but is used again whenever code is added or modified in the class to ensure that the changes have not broken anything.
